



Degree Project in Computer Science and Engineering

First level, 15 credits

Comparison of Pathfinding Algorithms in Dynamic and Congested Environments

A Comparative Analysis of

D* Lite and LPA*

BIRUK DEBEBE KEBEDE, JACOB FRIEDRICH

Comparison of Pathfinding Algorithms in Dynamic and Congested Environments

A Comparative Analysis of
D* Lite and LPA*

BIRUK DEBEBE KEBEDE, JACOB FRIEDRICH

Degree Project in Computer Science and Engineering, First Cycle 15 credits

Date: 2025-06-09

Supervisor: Johan Jansson

Examiner: Pawel Herman

Swedish title: Jämförelse av vägsökningsalgoritmer i dynamiska och trånga miljöer

School of Electrical Engineering and Computer Science

Abstract

The use of autonomous systems is rapidly expanding across various industries, increasing the need for efficient navigation in dynamic and complex environments. Handling frequent changes and multiple agents poses significant challenges. In this study, D* Lite and a modified version of LPA* (Lifelong Planning A*), adapted to suit the needs of the research, are compared in the context of a dynamic grid-based environment involving both single and multiple agents. The objective is to evaluate how well these algorithms handle frequent environmental changes, such as obstacles that appear out of nowhere and congestion.

In order to gather data, a simulation based methodology was used where the algorithms were implemented in Python using a pseudocode from academic papers. Pygame was used to simulate a 2D environment for visualization but most of the testing was done without it as it introduced additional computational overhead. The simulation modeled dynamic environments and congestion scenarios with varying agent counts. Data on replanning time, path length, computation time and fail rates was then collected for analysis.

The results showed that LPA* often finds short paths but struggles to handle fast changes. In contrast, D* Lite adapts more quickly and is more reliable, especially when multiple agents navigate simultaneously or when obstacles change frequently. The surprising finding at first was that D* Lite produced longer paths on average compared to LPA* but that was due to it failing slightly less often since it has an efficient replanning. However, both algorithms struggled when multiple agents blocked each other and performed poorly overall in rapidly changing environments. In summary, although LPA* was able to compete greatly, D* Lite is better suited for dynamic and crowded environments because of its ability to adjust faster.

Keywords

Pathfinding, global pathfinding, local pathfinding, incremental search, LPA* (Long life planning), D* Lite, static environment, dynamic environment, heuristic

Sammanfattning

Användningen av autonoma system ökar snabbt inom olika branscher, vilket ökar behovet av effektiv navigering i dynamiska och komplexa miljöer. Att hantera frekventa förändringar och flera agenter utgör betydande utmaningar. I denna studie jämförs D* Lite och en modifierad version av LPA* (Lifelong Planning A*), anpassade för att passa forskningsbehovet, i en dynamisk rutnätsbaserad miljö med både enskilda och flera agenter. Syftet är att utvärdera hur väl dessa algoritmer hanterar frekventa förändringar i omgivningen, såsom hinder som plötsligt dyker upp och trängsel.

För att samla in data användes en simuleringsbaserad metodik där algoritmerna implementerades i Python utifrån pseudokod hämtad från vetenskapliga artiklar. Pygame användes för att visualisera en tvådimensionell rutnätsmiljö, men testerna genomfördes utan denna visualisering eftersom den medförde extra beräkningskostnader. Simuleringen modellerade dynamiska miljöer och trängselscenarier med varierande antal agenter. Data om omplaneringstid, väglängd, beräkningstid och felfrekvens samlades sedan in för analys.

Resultaten visade att LPA* ofta hittar korta vägar, men har svårt att hantera snabba förändringar. D* Lite däremot anpassar sig snabbare och är mer tillförlitlig, särskilt när flera agenter rör sig samtidigt eller när hinder förändras frekvent. En oväntad observation var att D* Lite genererade längre vägar i genomsnitt jämfört med LPA*, men detta berodde på att den misslyckades sällan tack vare sin effektiva omplanering som inte kräver mycket tid. Båda algoritmerna hade dock problem att hantera flera agenter som blockerade varandra och presterade generellt dåligt i miljöer med mycket snabba förändringar. Sammanfattningsvis, även om LPA* presterade mycket bra, är D* Lite bättre lämpad för dynamiska och trånga miljöer eftersom den anpassar sig snabbare.

Nyckelord

Vägplanering, global vägplanering, lokal vägplanering, inkrementell sökning, LPA* (Lifelong Planning A*), D* Lite, statisk miljö, dynamisk miljö, heuristik

Table of Contents

1 Introduction.....	1
1.1 Problem Statement.....	1
1.2 Scope and Delimitation.....	2
1.3 Outline.....	2
2 Background.....	3
2.1 Pathfinding Fundamentals.....	3
2.2 Incremental Search Algorithms.....	5
2.3 LPA*.....	5
2.4 D* Lite.....	6
2.5 Related Work.....	7
3 Method.....	9
3.1 Implementation.....	9
3.2.1 LPA*.....	9
3.2.2 D* Lite.....	11
3.3 Simulation and Tests.....	13
3.4 Data Gathering.....	16
4 Results.....	17
4.1 Path Length in Dynamic Environments.....	17
4.2 Computation Time Under Environmental Changes.....	19
4.3 Replanning and Failure Rate.....	20
4.4 Performance in Multi-Agent Scenarios.....	21
5 Discussion.....	24
5.1 Interpretation and Analysis.....	24
5.2 Choice of Methods.....	25
5.3 Implementation Challenges.....	26
5.4 Limitations and Future Work.....	27
6 Conclusion.....	28
References.....	29
Appendix A.....	31

Chapter 1

Introduction

In recent decades, technological progress has transformed the way we live, work, and communicate. From smartphones and autonomous cars to warehouse automation, the integration of intelligent systems into everyday life continues to accelerate. At the heart of this transformation lies robotics, where machines are progressively gaining the ability to perform complex tasks independently, with less human intervention [1].

One important aspect of robotics is efficient navigation through an environment. This problem is known as pathfinding, which is not limited to robotics, but is also crucial in areas such as GPS navigation, video game AI, and network routing [2], [3]. To address this challenge, a variety of pathfinding algorithms have been developed. In static setting where the map does not change, well-known algorithms such as Breadth-First Search, Dijkstra's and A* are commonly used to compute an optimal path [4], [5].

However, the real world is rarely static. Dynamic and unpredictable changes, such as moving obstacles or newly discovered terrain, can complicate path planning. This is especially true for autonomous systems such as warehouse robots [6]. Unlike some industrial robots that operate in fixed locations or follow predefined paths, warehouse robots must rely on navigation algorithms to adapt in real time and safely move through dynamic environments [6]. This is due to the presence of moving obstacles and human workers that introduces uncertainty. Thus, it is essential for warehouse robots to navigate efficiently, minimize travel time, and avoid dynamic obstacles. In such contexts, traditional static algorithms as they are fall short.

To overcome these limitations, both incremental and non-incremental heuristic search algorithms have been developed such as D* Lite, Lifelong Planning A* (LPA*), Repeated A*, Generalized Lifelong Planning A* (GLPA*) [7], [8], [9]. These algorithms are capable of replanning in response to new information, which makes them suited for dynamic scenarios. This paper will therefore explore dynamic pathfinding algorithms that address the limitations of static approaches in real-world scenarios with changing environments.

1.1 Problem Statement

Most existing studies focus on optimality, the development and analysis of new variants of pathfinding algorithms. That is to say there appears to be a lack of a

comparative analysis of dynamic algorithms, especially any version of LPA* and D* Lite when it comes to highly congested and frequently changing environments. Thus, addressing this gap is important for guiding algorithm selection in multi-agent systems and environments with frequent changes.

The goal of this study is to evaluate how effectively D* Lite and a modified version of LPA*, adapted to fit the research context, handle frequent environmental changes, including the sudden appearance of obstacles and varying levels of agent congestion. The aim is to deepen the understanding of how dynamic pathfinding algorithms perform in highly complex environments, which leads to the following research question:

How do D Lite and a custom-implemented LPA* compare in terms of replanning efficiency, path length, and computation time when planning paths in highly dynamic environments, and how does their performance vary with different levels of agent congestion?*

1.2 Scope and Delimitation

This study focuses on analyzing existing algorithms with modifications to fit the needs of our research context. This is because the original studies from which these algorithms were taken abstract many details, particularly LPA*. Therefore, we have filled in these missing components, as well as added additional modifications that allows an agent to move in the way specified in the delimitation below.

The delimitation of this research is limited to a manually created 50×50 two-dimensional (2D) grid with varying obstacles. The grid is divided into cells and the agent can navigate to any of the four adjacent available cells. The environment is assumed to be partially known and the agent is capable of moving around, scanning its surroundings, and detecting changes in its immediate neighboring cells. It is also important to mention that this study does not attempt to simulate real-world physics or agent behavior beyond grid-based movements.

1.3 Outline

Chapter 2 provides the necessary theoretical background. It starts with fundamental concepts in pathfinding, introduces the concept of incremental search algorithms, and describes LPA* and D* Lite. Chapter 3 describes the experimental methodology, simulation environment, and the metrics used to evaluate their performance. Chapter 4 presents the results, followed by Chapter 5, which contains the discussion and suggestions for future work. Finally, Chapter 6 concludes the thesis with a summary and conclusions.

Chapter 2

Background

This chapter provides the theoretical foundation necessary to understand the context and contributions of this thesis. It begins with fundamental concepts in pathfinding, distinguishes between static and dynamic environments, introduces the concept of incremental search algorithms, and provides detailed descriptions of the LPA* and D* Lite algorithms. Finally, it discusses related work and further justifies the specific focus of this study.

2.1 Pathfinding Fundamentals

Pathfinding aims to determine the shortest and most efficient path between two points, as well as avoiding and preventing collisions with moving obstacles [10]. When it comes to environmental information, there are two important approaches, namely global and local path planning [11]. Global path planning refers to the action of finding the optimal route in a known environment, typically, a static setting where obstacles remain still and unchanged. In contrast, dynamic settings refers to environments where obstacles are unpredictable, and in such cases these algorithms may perform poorly or even fail.

On the other hand, local path planning is designed for unknown or partially known surroundings. It essentially, according to [10], works by constructing a local map using a sensor equipped mobile robot to gather information on the local surroundings, which is then used to adjust the path planning as shown in *Figure 2.1*. In order to achieve that, the agent must be aware of its own position, as well as the positions of other nearby objects. As a result, a suitable representation of the space, often referred to as the *configuration space* (e.g., a grid map), should be chosen [12].

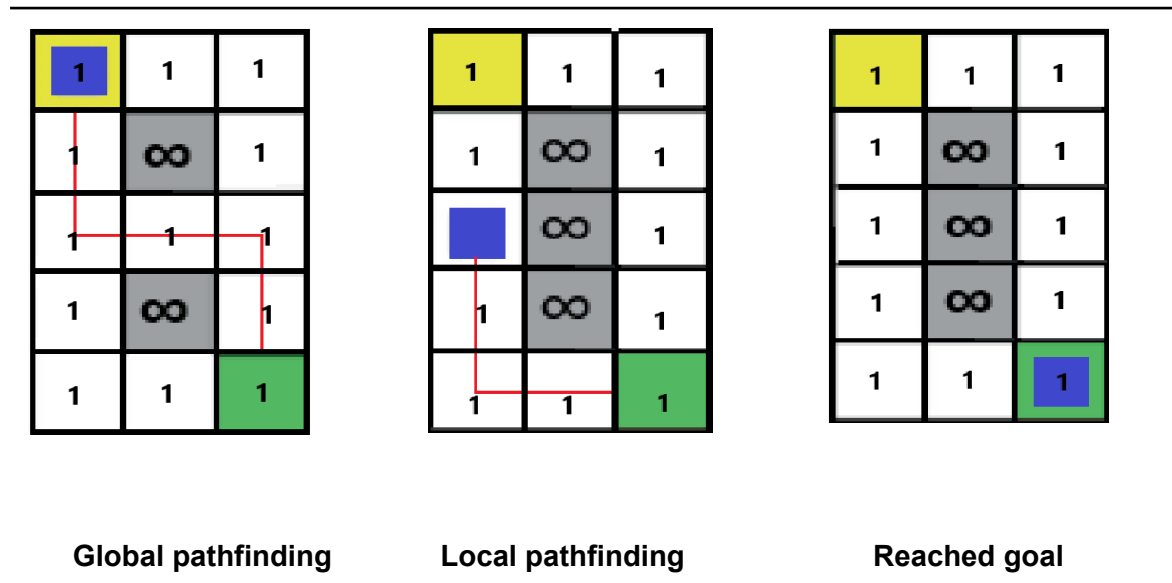


Figure 2.1: The picture depicts an agent's state in different scenarios in the environment. The symbols inside each cell indicate the edge costs associated with moving into that cell from any of its four adjacent cells. White cells with the number 1 inside correspond to traversable paths. Grey cells with infinity symbols indicate obstacles, representing nodes with infinite edge costs, i.e., they are impossible to enter. The yellow and green cells mark the start and goal cells, respectively. The blue square represents the agent's current position.

When dealing with static environments, algorithms like Dijkstra's or A* are well suited for finding the best paths [5], [13]. Among these, A* is especially notable for incorporating a heuristic, which is a knowledge based on approximations of the goal distance that help focus the search and solve problems faster than uninformed search methods [13]. The choice of heuristic depends on the environment, with common examples being the Manhattan distance and Euclidean distance [13]. These heuristics differ in how they measure distance. Manhattan distance sums horizontal and vertical steps, while Euclidean distance measures straight-line distance, as shown in Figure 2.2. However, many real-world scenarios involve dynamic conditions where obstacles can appear, disappear, or change their properties unexpectedly. Pathfinding in these environments requires algorithms capable of adapting to such changes.

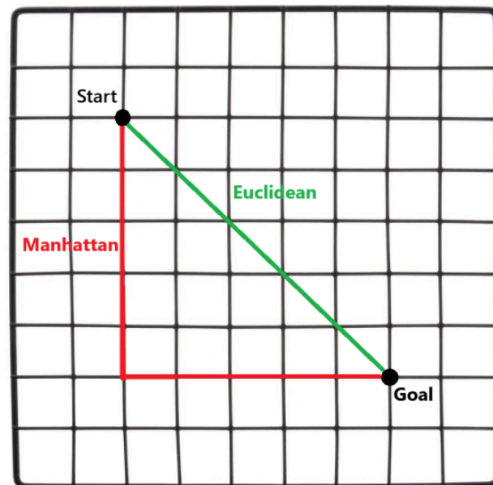


Figure 2.2: Illustration of Euclidean and Manhattan heuristics. The green straight line (as-the-crow-flies) represents the Euclidean heuristic: the distance from the start to the goal. In contrast, the red lines represent the Manhattan heuristic, which sums the horizontal and vertical movements from the start to the goal.

2.2 Incremental Search Algorithms

Traditional pathfinding algorithms like A* typically compute a path based on the information available at the start of the planning process. If the environment changes after the initial path is computed, these algorithms often need to replan completely.

To address such limitations, incremental search algorithms update or repair previously computed solutions in response to changes in the environment [7], [8]. Rather than discarding the already existing plan, they identify parts of the plan affected by the change and reuse information from the previous search effort to minimize computational overhead. LPA* and D* Lite are some examples of incremental heuristic search algorithms designed for dynamic environments [7], [8].

2.3 LPA*

LPA*, developed by Koenig, Likhachev and Furcy [14] in 2001, is an incremental heuristic search algorithm that extends A*. It is designed to handle dynamic environments where obstacles may appear and edge costs can change over time, while the start and goal nodes stay fixed. LPA* keeps track of cost estimates from the start to all nodes and uses a priority queue to efficiently update affected nodes when edge values change.

To achieve this, LPA* maintains two key values for each node, $g(s)$, which represents the current best-known cost from the start node to any node s , and $rhs(s)$ (right-hand side), a one-step lookahead estimate calculated from the g -values of neighboring nodes and the edge costs [14]. A node is said to be locally consistent when $g(s) = rhs(s)$, and inconsistent otherwise. The goal of the algorithm is to resolve inconsistencies by updating $g(s)$ to match $rhs(s)$ whenever inconsistencies are detected, allowing it to respond to changes in the environment.

The algorithm uses a priority queue to manage locally inconsistent nodes, prioritizing them based on a lexicographic key that combines their g -value, rhs -value, and a heuristic estimate h . In each iteration, LPA* selects the node with the smallest key and makes it locally consistent by setting $g(s) = rhs(s)$ when appropriate. This update can cause neighboring nodes to become inconsistent, prompting further updates. When an edge cost changes, LPA* propagates the impact by updating the rhs -values of affected nodes, which in turn updates the corresponding g -values, ensuring the path is valid.

2.4 D* Lite

The D* Lite algorithm, also presented by Koenig and Likhachev [8], builds upon the concepts of LPA* but focuses more on enabling smooth agent movement. Although D* Lite builds upon the concept of LPA*, it also draws inspiration from the earlier D* algorithm introduced by Stentz in 1994 [15], hence the name "*D* Lite*". According to [8], D* Lite performs at least as well as the original D* algorithm but with significantly less complexity.

Similar to LPA*, D* Lite uses $g(s)$ and $rhs(s)$ values and a priority queue. However, it initially performs its search backwards from the goal node s_{goal} towards s_{start} . Here, $g(s)$ represents the estimated best cost from s to s_{goal} , and $rhs(s)$ is a one-step lookahead estimate using neighbors' g -values and edge costs. Nodes are prioritized in the queue based on a two-part key: $k(s) = [k_1(s); k_2(s)]$.

$$k_1(s) = rhs[s] + h(s, s_{start}) + key_m$$

Through a key modifier (key_m), a mechanism that helps to maintain priority values for nodes in the priority queue, changes in priority are tracked by carrying over key_m throughout the navigation process. The heuristic h estimates the cost from s_{start} to a given node s . The value of $k_1(s)$ is calculated by summing the three values shown in the formula above. Since multiple nodes in the queue can have the same k_1 value, it

might result in the same sum: $k_1(s_1) = k_1(s_2)$. In such cases, the second formula below is used as the priority.

$$k_2(s) = g[s] + rhs[s]$$

In both D* Lite and LPA*, blocking a path is equivalent to setting the cost between two nodes to infinity. Note that it is the edge cost (not the node) that determines traversability. An infinite cost means the edge is blocked, while any finite cost implies that a path exists. Moreover, D* Lite requires a partial knowledge of the environment and when an edge cost change (e.g., due to a newly detected obstacle), D* Lite updates the relevant nodes and recomputes the shortest path by expanding nodes until $s_{start} = s_{goal}$ [8]. This backwards search and the incremental updates allow the agent to efficiently replan its path from its current location as it moves and discovers new information.

2.5 Related Work

There are several articles that compare D* lite with different types of algorithms. In [10], the authors proposed a new algorithm called *Conflict Based Search with D* (CBS-D*)*, which is a hybrid approach that along with D* Lite focuses on preventing collisions with dynamic obstacles. Unlike plain D* Lite, which requires partial knowledge of the environment, this algorithm is better suited for unknown environments. The initial assumption before the experiment was that D* Lite provides superior performance when the map changes slightly. In contrast, CBS-D* was expected to effectively anticipate and avoid dynamic obstacles in large, complex, and unknown environments but that it might perform badly when the environment changes unexpectedly. The results showed that CBS-D* in fact does outperform D* Lite in terms of pathfinding accuracy and obstacle avoidance efficiency [10].

In another study [16], researchers presented an algorithm called Multipath Adaptive A* (MPAA*), where they compared to D* lite. MPAA* is based on the concept of reusing paths from previous A* searches to improve efficiency during replanning. The results showed that MPAA* was on average faster than D* Lite and that it performs over three times faster on random maps. However, D* Lite demonstrated better performance in scenarios where only a few optimal paths existed. This aligns with findings from another study [8] where researchers noted that the D* Lite algorithm does not handle unknown environments effectively.

Despite advances in algorithmic research, including the growing use of machine learning approaches in Multi-Agent Path Finding (MAPF) [17], there is still a gap in performance comparisons of dynamic algorithms. In particular, little work has

explored how these algorithms, especially any versions of D* Lite and LPA*, perform in highly dynamic environments and across varying levels of agent density.

Therefore, this study aims to contribute with empirical data to fill this gap, in hope of providing some valuable insight for practitioners (eg., warehouse robotics operators) selecting or building upon algorithms for dynamic settings with multi-agent systems.

Chapter 3

Method

This chapter presents the methodology of our study. It starts by explaining the implementation of the algorithms, then describes the simulation environment. Finally, we discuss the testing procedures and the metrics used to evaluate performance.

3.1 Implementation

This research uses various resources, including pseudocode from research papers [8], [14], Python libraries such as Pygame, collections (defaultdict), heapq, and threading. The heapq module is used for priority queue management, allowing efficient retrieval of the lowest-cost node during pathfinding. Pygame supports visualization by rendering a 2D grid where agents move based on the computed paths. To enable simultaneous movement of multiple agents, threading is used, allowing concurrent agent updates and interaction within the environment.

In order to align with our research question, we made several of our own design decisions. For instance, instead of having global or any other type of awareness of the environment, each agent is limited to sensing changes only within its four adjacent cells (up, right, left, and down). If there is an obstacle on a node, the cost (edge cost) to reach it is considered infinite. This constraint reflects a localized sensing limitation, where agents can only perceive what is directly in front of them.

3.2.1 LPA*

LPA* follows a forward search strategy, which means it repeatedly searches from the start node to the goal as mentioned in *section 2.3*. The algorithm was implemented using research papers from Koenig [14]. The original pseudo code for LPA* does not describe the movement of an agent and much of the details is abstracted away or explained in the report, for instance the procedure for extracting a path based on the computed g-values. Therefore, to implement LPA*, we saved the found path into a list by iterating through the g-values from neighbor to neighbor. Whenever there is a change in the environment, it clears the list and adjusts its path to the new environment. In order to make LPA* suitable for our research, we modified it so that the cell where the agent fails to advance is treated as a start cell and its rhs-value is set to 0, which allows the agent to start replanning from it, instead of the start. To maintain the validity of the priority queue, the elements were iterated through, and

the rhs-value of the current node was subtracted from their priorities to retain the correct relative order. Setting the rhs of the current node to 0 caused its neighbors to have lower key values, but by adjusting the queue accordingly, we helped avoid incorrect prioritization.

The *calculateKey* function in the following pseudocode is responsible for computing the priority of nodes based on their cost and heuristic. The *computeShortestPath* function processes nodes from the priority queue to update their cost estimates. The *updateVertex* function manages the priority queue and updates the rhs values of nodes. The main function ties everything together, repeatedly executing the logic until a valid path to the goal is found. Additionally, the *scanEnvironment* function checks the four neighboring cells to detect any changes in the environment that may affect the path. Lastly, *createPath* returns a list of the extracted path. See Appendix A for full implementation.

```
function calculateKey(s):
    return (min(g[s], rhs[s] + h(s, goal)), min(g[s], rhs[s]))

function initialize()
    U = empty
    for all s in neighbors rhs(s) = g(s) = 0
    rhs(start) = 0
    insert (calculateKey(u), u) into U

function updateVertex(u)
    if u ≠ start then rhs(u) ← min_{s ∈ neighbors(u)} [cost(u, s) + g(s)]
    if u is in U then remove u from U
    if g[u] ≠ rhs[u] then insert (calculateKey(u), u) into U

function computeShortestPath()
    while (top of U < calculateKey(goal) OR rhs[goal] ≠ g[goal]):
        (key, u) ← remove top of U
        if g[u] > rhs[u]:
            g[u] ← rhs[u]
            for each s of neighbors[u] do updateVertex(s)
        else:
            g[u] ← ∞
            for each s in neighbors[u] + [u]:
                updateVertex(s)
    createPath()

function main(agent, num):
    loop forever:
        clear agent's path
        computeShortestPath()
        prev ← Null
        for each node in path:
            if prev is not null:
                start ← prev
```

```

        mark matrix[prev.x][prev.y] as empty and value  $\leftarrow$  0
        if node == goal:
            Path has been found
        if matrix[node.x][node.y] == 0:
            mark matrix[node.x][node.y] as part of path
            prev  $\leftarrow$  node
            agentPrev  $\leftarrow$  node
        else if matrix[node.x][node.y] > 0:
            stay at previous node
            break out of inner loop
    start  $\leftarrow$  prev
    scanEnvironment(prev, agent)
    rhs[start]  $\leftarrow$  0
    update the priority of all nodes inside U
    updateVertex(prev)

```

3.2.2 D* Lite

D* Lite uses a backward search strategy. Backward search strategy means that the global search part of the algorithm creates its initial path by calculating from goal to start. While following that path, if a change occurs, it will adjust its values as previously discussed in *section 2.4*, which allows the agent to move effectively. The main code skeleton was derived from the research paper [8], written by the algorithm founders Koenig and Likhachev. It is also worth noting that this research uses the optimized D* Lite which was presented on the same paper [8], which, among other improvements, reduces unnecessary queue operations.

In terms of implementation, the main method is where the actual navigation occurs: it initializes the agent, computes the initial path, and moves the agent step by step towards the goal while reacting to changes in the environment. The function *calculateKey* handles the priority of a node *s* (see *section 2.4*). The *updateVertex* function ensures that node *u* is correctly placed in the queue if it's inconsistent, or removes it if it becomes consistent. Meanwhile, *computeShortestPath* together with *scanEnvironment* continuously processes nodes to find and maintain the shortest path.

```

function calculateKey(s)
    return min((g[s], rhs[s]) + h(start, s) + km, min(g[s], rhs[s]))

function updateVertex(agent, u)
    if g[u]  $\neq$  rhs[u] and  $u \in U$  then update U with calculateKey(u)
    else if g[u]  $\neq$  rhs[u] and  $u \notin U$  then insert calculateKey(u) into U
    else if g[u] == rhs[u] and  $u \in U$  then remove u from U

function computeShortestPath(agent)
    while (top(U).key < calculateKey(start) or rhs[start] > g[start]) do
        k_old, u  $\leftarrow$  top of U

```

```

    k_new ← calculateKey(u)

    if k_old < k_new then
        update U with (k_new, u)
    else if g[u] > rhs[u] then
        g[u] ← rhs[u]
        remove u from U
        for each s in neighbors[u] do
            if s ≠ goal then rhs[s] ← min(rhs[s], cost[(s, u)] + g[u])
            updateVertex(s)
    else
        g_old ← g[u]
        g[u] ← ∞
        for each s in neighbors[u] ∪ {u} do
            if rhs[s] ≠ u and rhs[s] == cost[(s, u)] + g_old then
                if s ≠ goal then
                    rhs[s] ← min_{s' ∈ neighbor(s)} [cost(s, s') + g(s')]
            updateVertex(s)

function scanEnvironment(u, agent)
    for each v in neighbors[u] do
        if blocked neighbor has been freed then newCost ← 1
        else if free neighbor has been blocked then newCost ← ∞

    if there was any change
        km ← km + h(last, start)
    last ← start

    for each n in neighbors[v] do
        cost[(n, v)] ← newCost
        cost[(v, n)] ← newCost
        c_new ← newCost
        c_old ← cost[(n, v)]

        if u ≠ goal then
            c_old ← cost[(n, v)]
            if c_old ≥ c_new then
                rhs[u] ← min(rhs[u], newCost + g[v])
            else if rhs[u] ← c_old + g[v] then
                rhs[u] ← min_{s' ∈ neighbor(u)} [cost(u, s') + g(s')]
        updateVertex(u)
    computeShortestPath()

function main (agent)
    last ← start
    initialize
    computeShortestPath()
    while start ≠ goal do
        if(rhs(start) == ∞) then there is no known path
        nextMove ← pickMin(start)
        if matrix[nextMove.x][nextMove.y] == 0 then

```

```

tempPos ← pos
pos ← nextMove
start ← nextMove
matrix[nextMove.x][nextMove.y] ← 1
matrix[tempPos.x][tempPos.y] ← 0
scanEnvironment(start, agent)
matrix[start.x][start.y] ← 0

```

3.3 Simulation and Tests

To visualize the algorithm and create a simulation, the Pygame library was used. We relied on a custom-implemented 50×50 grid to generate a 2D environment as specified in the scope (section 1.2). The environment representation is partially inspired by [8], but it is specifically designed to resemble a warehouse layout. It reflects a typical environment where autonomous robots must navigate efficiently around aisles and storage shelves, which serve as obstacles. The threading library was utilized to enable multiple agents to operate simultaneously within the grid, allowing each agent to run on a separate thread and create a more realistic simulation. The agents' overall goal was to navigate in the environment shown in *Figure 3.1*, moving toward their assigned destinations while avoiding all types of obstacles along the way.

It is important to note that obstacles encompass not only walls (i.e., blocked cells) but also other agents, which act as dynamic obstacles. A collision, in the context of this study, refers to an agent coming into contact with a dynamic obstacle, such as a cell that becomes blocked during navigation or another agent. To prevent collisions between agents, the simulation enforces that no two agents can occupy the same cell simultaneously. Essentially, when an agent occupies a cell, that position is perceived by others as blocked, just like a newly introduced wall. Agent collisions are therefore handled in the same way as blocked cells by treating any cell occupied by another agent as impassable during path planning.

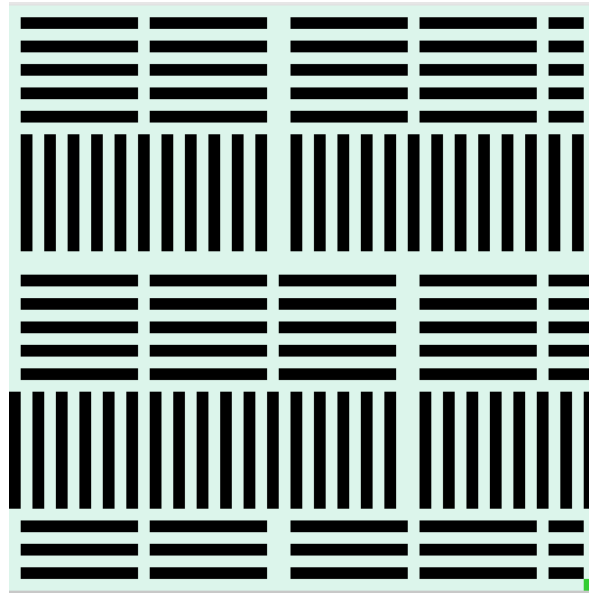


Figure 3.1: The unaltered environment in which the agents using both algorithms will run on. The agent starts at the upper-left corner, and its goal is the green-marked cell at the lower-right corner.

The first part of testing involves letting one agent navigate through the grid shown in *Figure 3.1* at 60 frames per second (fps), while the environment changes every 0.01 to 0.1 seconds. The grid is divided into four parts as shown in *Figure 3.2* because the environmental updates occur randomly, and we wanted to ensure that those updates are spread across the entire grid. For example, every 0.1 s, a cell in each of the four quadrants will be flipped from free to blocked, and another from blocked to free. This approach prevents certain areas from being disproportionately affected, which could otherwise result in either no blocked paths or all paths being blocked for the navigating agent. Alternating between turning cells off and on is necessary because turning cells off only would eventually congest the path and block critical routes. As a result, even if the agent can discover alternate paths, it will be unable to reach its goal. The initial map for both algorithms was the same as shown in *Figure 3.1*, but since the changes were random, the agents encountered different obstacles even though efforts were made to keep conditions fair (see *section 5.3*).

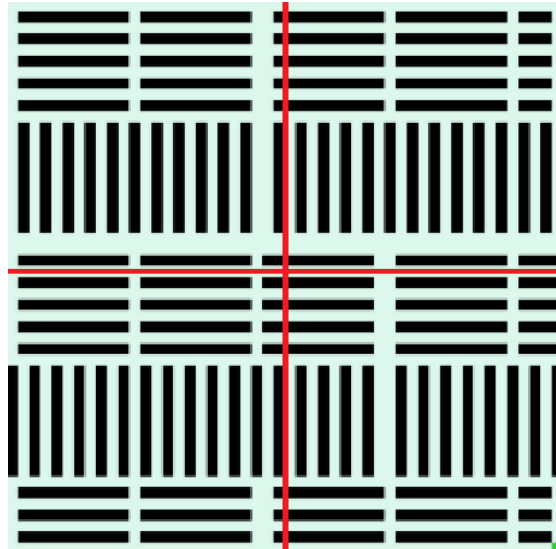


Figure 3.2: The environment after it has been divided into 4 quadrants so the random changes made are spread throughout the map.

For the second part of the testing, multiple agents were introduced into the environment, varying in numbers: 10, 20, 30 or 40. For each agent in the given set, a thread is created with their assigned starting point initialized, but they are not allowed to start immediately. Once the threads for all agents are created, they are all started simultaneously. To ensure congestion, the agents must begin in the upper half of the grid as shown in *Figure 3.3*, so no agent starts within one cell of the goal. Additionally, to test the congestion even more, there is only one goal destination for all agents.

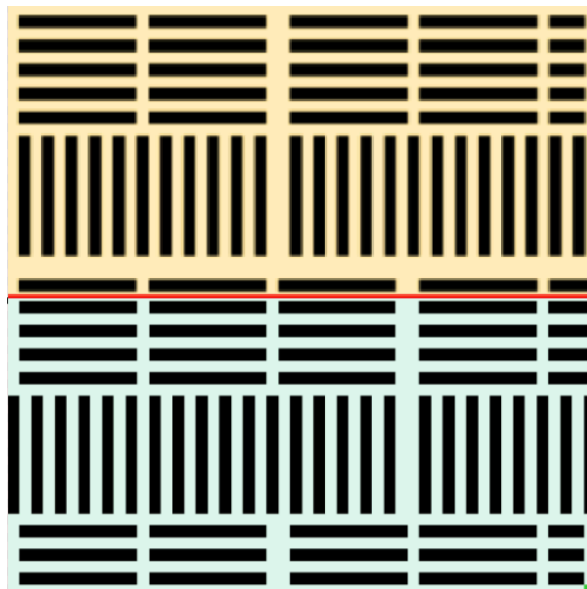


Figure 3.3: The upper yellow part of the grid is the only area where agents will be created during the congestion testing.

3.4 Data Gathering

This study evaluates the algorithms using several key metrics, including computation time and replanning time, which capture different aspects of performance and are thus important to distinguish. Computation time refers to the total time taken by an agent from the beginning of its execution to the moment it reaches the goal. This includes the initial path planning and all subsequent replanning events due to dynamic changes in the environment. In contrast, replanning time specifically refers to the duration between the detection of an obstacle and the moment an agent resumes movement with an updated path.

In order to gather data for the first test scenario, the algorithms were run separately with varying rates of environmental change, performing 30 runs for each. Then the failure rate, mean path length (the number of cells traversed by the agent), mean computation time, and the standard deviation was calculated for both algorithms. This process was repeated for each update frequency as mentioned in *section 3.3*. It is also important to note that the obtained mean path length and computation time values include only successful trials, excluding failed attempts from the analysis.

For the next phase of testing, which focused on congestion, we began with 10 agents for each algorithm, running at 60 fps. In this case, the success rate was a more important metric to collect than time or path length as the placement of the agents is random. However, this testing consumed significant time and CPU resources, and as congestion increased, it began to lag. As a result, the samples were limited to 10 per algorithm for each group size.

Lastly, to test the replanning time in both cases, the elapsed time in seconds from the moment an obstacle was first detected until the algorithm resumed running was measured.

Chapter 4

Results

This chapter presents the results of the comparative analysis of the two path finding algorithms based on path length, computation time, and behavior under dynamic changes and congestion. Both numerical results and visual illustrations are provided below.

The initial unaltered environment through which the agents navigate is shown in *Figure 4.1*, while *Figure 4.2* depicts the environment shortly after the simulation begins, illustrating how it changes over time. These changes happen at varying update frequencies, where eight cells are modified at each update (see section 3.3). The blue, green and black squares represent the agent, the goal and obstacles, respectively. The remaining non-black regions represent the available paths accessible to the agent.

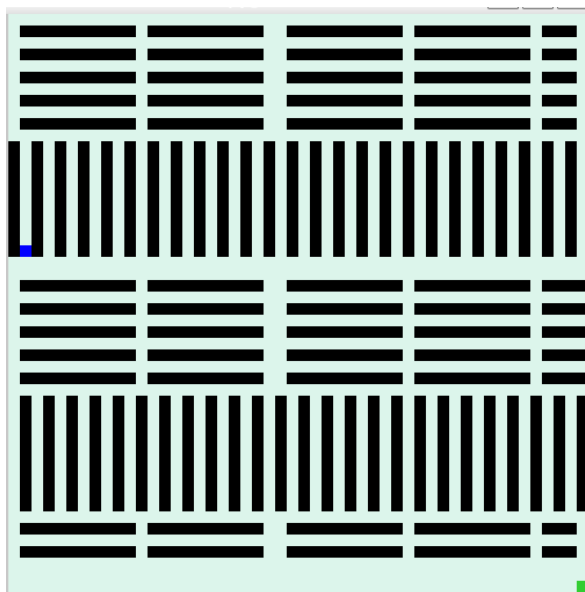


Figure 4.1: The main environment that the algorithms will be run on.

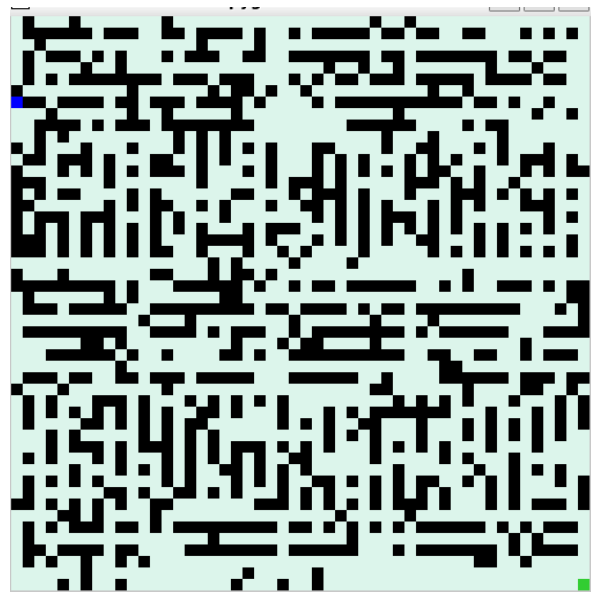


Figure 4.2: The environment after it has been modified while the agent is still moving.

4.1 Path Length in Dynamic Environments

The mean path length differences when a single agent navigates a 50×50 grid with dynamic obstacles are presented in *Table 4.1* and illustrated in *Figure 4.3*. As shown

in the table, LPA* achieves shorter mean path length than D* Lite by 7.62%, 12.3%, 5.1%, 8.1%, 4.5%, and 2.3% at environmental update intervals of 0.03s, 0.04 s, 0.05 s, 0.06 s, 0.08 s, and 0.09 s, respectively. D* Lite only performs better in one case, achieving a shorter mean path length at an environmental update interval of 0.07 s. It is also evident that D* Lite generally has a higher standard deviation compared to LPA*, which suggests that LPA* is more consistent. However, it is important to note that the obtained mean path length and computation time values are based only on successful trials. LPA* failed to produce results in many scenarios involving significant environmental changes, which means the averages reflect only the less challenging cases where the algorithm was able to succeed.

It is also worth noting that data for both D* Lite and LPA* at an environmental update interval of 0.01s and 0.02s is not included. That is because their performance becomes progressively worse under rapid environmental changes, making it difficult to obtain reliable results.

Table 4.1: Mean path length (number of grid cells) and the standard deviation calculated from 30 successful samples for agents using the LPA* and D* Lite algorithms under varying rates of environmental change.

Environmental change rate per second	Mean path length (D* Lite)	Mean path Length (LPA*)	Standard deviation (D* Lite)	Standard deviation (LPA*)
0.01	Fail	Fail	Fail	Fail
0.02	Fail	Fail	Fail	Fail
0.03	114.1	105.4	22.7	10.58
0.04	116.7	102.3	18.07	7.9
0.05	108.3	102.8	12.49	9.01
0.06	108.8	100	22.53	2.92
0.07	100.53	101.4	4.98	5.61
0.08	105.24	100.5	11.18	3.83
0.09	100.52	98.2	5.12	0.7
0.1	100.3	100.3	5.08	7.29

Comparison of Path Lengths for LPA* and D* Lite

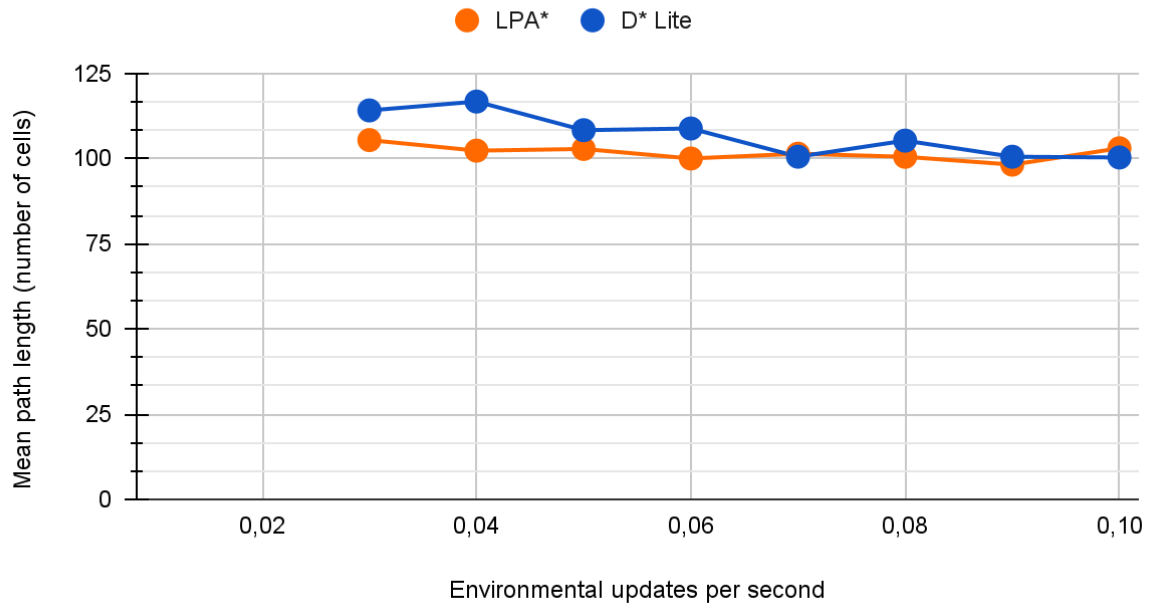


Figure 4.3: The mean path length for an agent using D* Lite and LPA* in the same grid with similar environmental changes.

4.2 Computation Time Under Environmental Changes

As mentioned in section 4.1, a single agent navigates a 50×50 grid with dynamic obstacles. The results of the mean computation time are presented in *Table 4.2* and illustrated in *Figure 4.4*. Overall, LPA* performed better in terms of mean computation time across all intervals, except at an environmental update interval of 0.1 s, where both algorithms showed similar performance, as the environment was nearly static at that rate. Notably, the standard deviation for D* Lite is higher than that of LPA*.

Table 4.2: Mean computation time performance (in seconds) and the standard deviation calculated from 30 successful samples for agents using LPA* and D* Lite algorithms under varying rates of environmental change.

Environmental change rate per second	Mean Time (D* Lite)	Mean Time (LPA*)	Standard deviation (D* Lite)	Standard deviation (LPA*)
0.01	Fail	Fail	Fail	Fail
0.02	Fail	Fail	Fail	Fail
0.03	2.05	1.73	0.4	0.17
0.04	2.15	1.69	0.39	0.13
0.05	2.22	1.71	0.36	0.15

0.06	2.12	1.66	0.44	0.06
0.07	1.87	1.68	0.11	0.101
0.08	1.94	1.68	0.19	0.068
0.09	1.91	1.65	0.11	0.07
0.1	1.66	1.68	0.084	0.14

Comparison of Computation Time for Pathfinding: LPA* and D* Lite

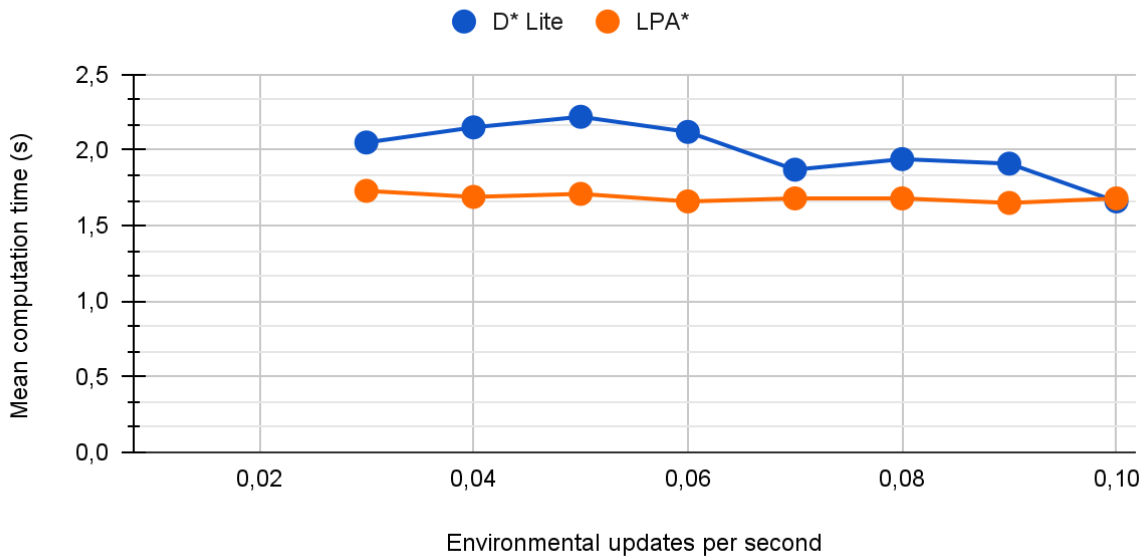


Figure 4.4: The mean computation time for an agent using D* Lite and LPA* in the same grid with similar environmental changes.

4.3 Replanning and Failure Rate

It is observable from *Table 4.3* that the replanning time of D* Lite is much faster than the replanning time of LPA* .

Table 4.3: Mean replanning time, based on 30 samples, for each algorithm when encountering an obstacle absent from the initial environment.

Algorithm	Mean Replanning time (s)	Standard deviation(s)
LPA*	0.01	0.0056
D* Lite	0.0015	0.002

The following diagram (*Figure 4.5*) presents the failure rate of the algorithms across different environmental update intervals, as discussed in *sections 4.1* and *4.2*. In this context, failure means that an agent is unable to find a valid path to the goal for any reason and becomes permanently stuck. The failure rate reflects the number of failed attempts out of 30 runs performed by a single agent for each scenario. Across all scenarios, the average number of failures for LPA* exceeds that of D* Lite.

Agent failure rate using D* Lite and LPA*

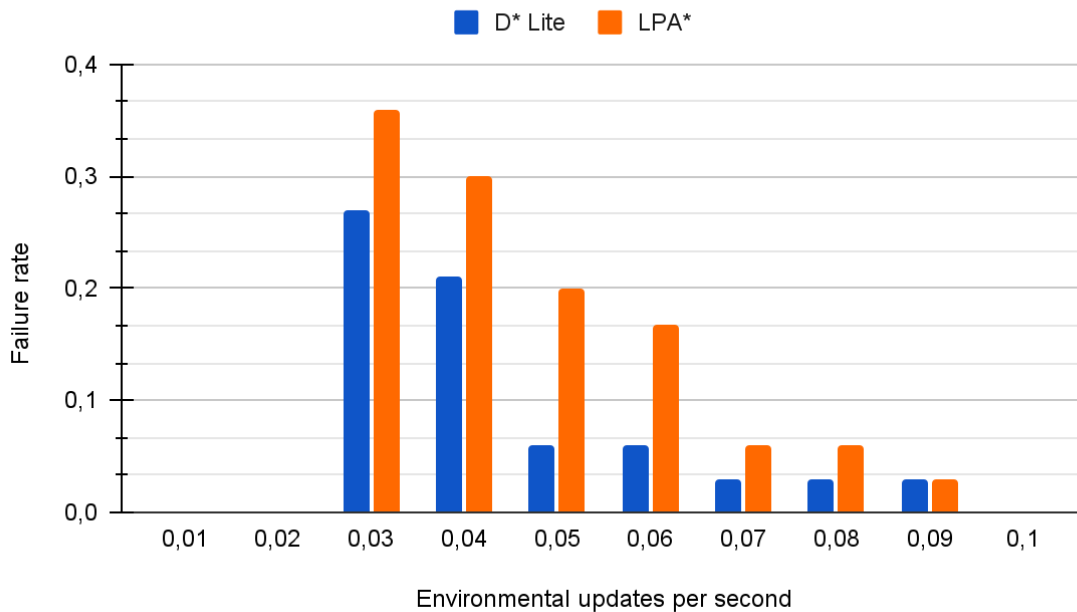


Figure 4.5 : Failure rate across scenarios. The x-axis shows the proportion of failed agents (30 runs), and the y-axis lists scenarios with different environmental change rates.

4.4 Performance in Multi-Agent Scenarios

As shown in *Figure 4.6*, the grid captures the state of the environment as multiple agents pass through it. The agents move at a speed of 60 frames per second (fps) in the simulation. The blue squares represent the agents, the green square represents the goal, and the black squares represent obstacles.

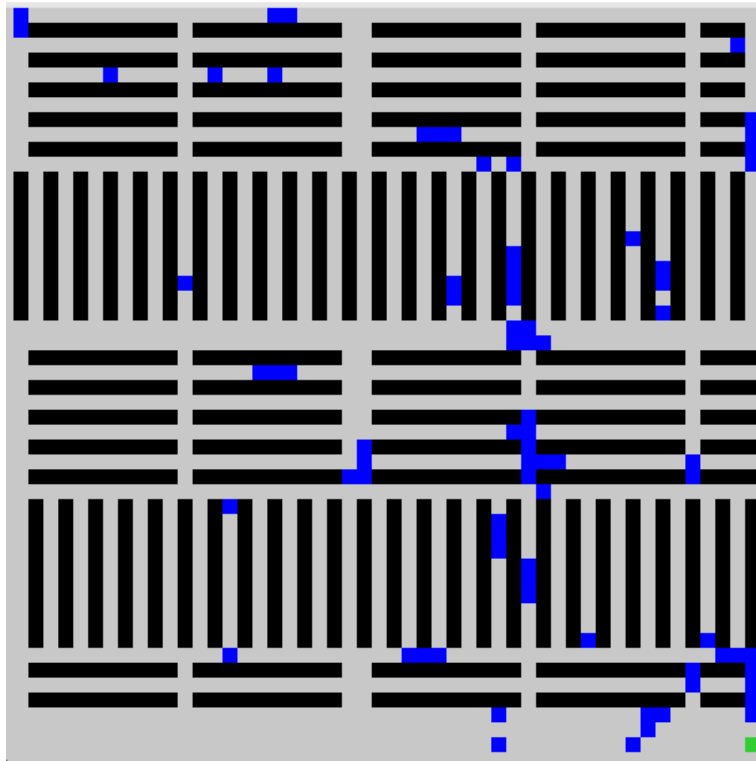


Figure 4.6: Visualization of multiple agents navigating through a 50×50 grid. .

The average number of agents that successfully found a path in a congested grid environment, where multiple agents are operating concurrently using different threads, is shown in *Figure 4.7*. A clear difference in performance emerges under these crowded conditions. D* Lite consistently outperforms LPA* in every case. In other words, almost all agents that used D* Lite successfully reached their destination in all the tests, while LPA* began to show worse results as the number of agents increased.

Performance of LPA* and D* Lite Under Congestion at 60 FPS

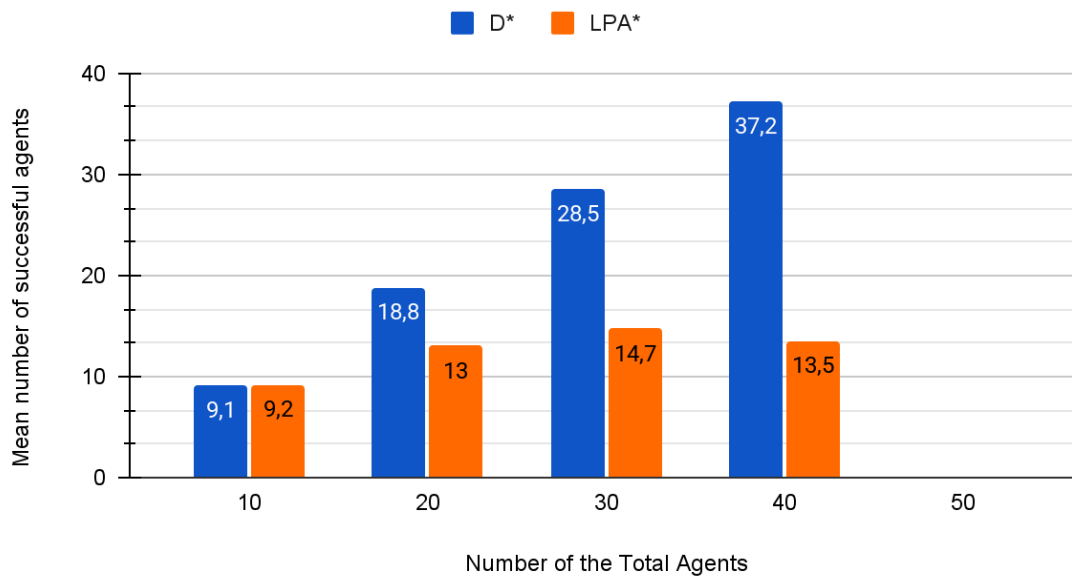


Figure 4.7: The average number of agents that successfully found a path as the number of agents increases..

Chapter 5

Discussion

The following chapter interprets the results in detail, justify the methodological choices made, and discuss the challenges, limitations, and potential directions for future work.

This study aimed to evaluate and compare the performance of LPA* and D* Lite in terms of replanning efficiency, path length, computation time, and success rate when navigating a dynamic 2D environment with varying number of agents. Overall, the results show that, on average D* Lite produced longer paths and required more computation time than LPA* as the environment changed. However, this is mainly because LPA* failed in many scenarios and only produced results when it was affected less by the environmental changes, thus lowering its average. When it came to handling congestion D* Lite was better in every scenario.

5.1 Interpretation and Analysis

When the environment changes every 0.1 s relative to the agent's speed, it is almost like a static environment and their performances are similar in both path length and time, which is not surprising, considering both use a version of A* for static pathfinding. That said, the results gave the impression that LPA* performed better than D* Lite in terms of path length and computation time in scenarios where an agent navigates through a rapidly changing grid. These findings seem to indicate that LPA* is more effective at handling dynamic obstacles. However, the findings in *section 4.3* suggest that D* Lite is not as inferior to LPA* as it might have initially appeared. Notably, *Figure 4.5* shows that LPA* has a higher failure rate compared to D* Lite. This is because LPA* tends to produce results primarily when the environment remains stable, whereas D* Lite demonstrates greater robustness by failing less often.

This resilience can be attributed to D* Lite's efficient replanning behavior, which increases its chances of eventually finding a path even if it takes longer time and it has to discover more cells. To elaborate, its faster replanning capability enables it to generate new paths, often before the map becomes unrecognizable, as shown in *Figure 4.2*. This makes it less likely to get stuck in narrow passages compared to LPA*. Replanning time is therefore a critical factor and one of the main reasons for

the higher failure rate of LPA*, as well as the observed differences in path length and computation time.

In our implementation of LPA*, each time a change is detected, the agent must clear and rebuild the list from its current position. This results in slower replanning, which in turn allows the environment to change even further before the algorithm can respond. As reflected in *Table 4.1* and *4.2*, D* Lite exhibited a higher standard deviation in both computation time and path length. This indicates its tendency to continue exploring in complex situations, often at the cost of a worse average completion time and path length, while LPA* generally fails. Although the bias introduced by using only successful trials made it hard to directly compare their mean path length and computation time, the values for these metrics along with the recorded failure rates indicate that D* Lite is the more robust algorithm.

Moreover, some of the findings in this research align with the studies mentioned in the background. For instance, one study [8] stated that D* Lite provides a great performance when the map changes slightly, which holds true in our case. When the environment changed at a significantly high rate as shown in *Tables 4.1* and *4.2*, it is observable that both D* Lite and LPA* failed to produce any results.

Another study [10] discussed in the background concluded that CBS-D* outperformed D* Lite in terms of obstacle avoidance efficiency and overall computational steps. This is also understandable, because although D* Lite performed better than LPA* when it comes to congestion. It sometimes still struggled with handling agent collisions, where agents became stuck or blocked one another. We believe this is because when two agents block each other, particularly if one agent has exhausted all possible options, it may assume there are no further paths forward. In an attempt to discover alternative routes, the agent might try to backtrack. However, if another agent is approaching from behind, this can potentially lead to a deadlock. Such a blockage may cause a chain reaction, where other agents following similar paths become stuck. The same observation applies to LPA*, but not only that, LPA* also struggles increasingly as more agents are introduced, largely due to the computational overhead (creating and clearing lists) discussed in this study, which worsens with a higher number of agents.

5.2 Choice of Methods

The reason a 50×50 grid was chosen was because any smaller grid would not effectively test congestion, and not to mention agents would not travel far enough to encounter changes of obstacles. A larger grid, on the other hand, would increase computational overhead due to higher memory usage. Additionally, using a grid with a rectangular layout simplifies environment representation through a matrix, allowing straightforward indexing and manipulation of cells. Agents can also navigate every part of the grid easily, which would be difficult to achieve with, for instance, a circular

shape due to its irregular boundaries and more complex spatial representation. The obstacle layout mimics a typical warehouse setting, making the simulation close to a real-world scenario.

In addition to the environment setup, agents moved at 60 fps, as this is generally the frame rate used by many real-time games. Not only does using 60 fps allow us to see the simulation as it happens, but it also gives us control over the timing of environmental updates. However, since the algorithms react to obstacle changes based on agent movement, the specific speed is not critical. What matters is ensuring that the agents have the same speed and are exposed to comparable changes in the environment.

5.3 Implementation Challenges

To maintain fairness, efforts were made to generate similar obstacle changes. One approach considered was using a fixed random seed to recreate identical scenarios. However, this introduced new challenges: if an agent fails in a particular scenario, recreating the exact same scenario would potentially cause that agent to repeatedly fail in every run, as it would be subjected to the same obstacles at the same time. Furthermore, a seed that results in success for one algorithm does not guarantee success for the other. Consequently, in our testing, the grid was divided into four quadrants where cells were alternately blocked and unblocked at set intervals to distribute environmental changes evenly and avoid complete blockage or overly free paths. However, this approach still introduced randomness that could unevenly affect agent performance, and despite efforts to balance changes, it does not guarantee a perfectly comparable scenario for both algorithms.

Additionally, in our implementation of LPA*, when the agent encounters an obstacle, the current node is set as start and its rhs-value to 0. To maintain a valid queue, the priorities of all other nodes in the priority queue were adjusted by subtracting their values as mentioned in *section 2.3*. This can affect the overall efficiency of the algorithm considering the fact that we have to iterate through the queue whenever an obstacle is met. On the other hand, it is to help us keep the incrementality of LPA* since it means that we do not discard other potential paths already present in the queue while only prioritizing the new search with lower priority value. That is because priority also depends on the rhs-value which in this case have been modified.

Nonetheless, this experiment remains valuable, as it offers a comparison between two dynamic algorithms in a complex environment. It highlights how design decisions, such as creating a list, for instance, can affect algorithm behavior and its replanning time. Moreover, by comparing two closely related algorithms under similar conditions, we gained a deeper understanding and insight into what is most important in a dynamic algorithm: the ability to replan. This led us to realize that in a

highly dynamic environment, although finding the shortest path quickly is ideal, it is more crucial to find any feasible path as quickly as possible.

In summary, one of the most important takeaways from this research is that, in environments such as a warehouse that constantly receives and ships goods, prompting frequent changes to the surroundings, implementers should prioritize the replanning time of any algorithm when making their choice.

5.4 Limitations and Future Work

Some of the limitations we faced included the fact that all testing was done on our personal laptops, which caused the simulation to lag for the most part. As more agents were added, the lag increased, so the maximum number of agents was limited to around 40. Although the algorithms, especially D* Lite could handle many more, this was not consistently achievable on our laptops. In the real world, each robot operates on its own machine, so actual performance may differ from what we demonstrated in this study.

Given these constraints, there is still room for further studies. For example, we only tested their performance on a single medium-sized grid, but future studies could examine their behavior in both larger and smaller environments to determine if, for instance, LPA* performs equally well in smaller grids. Additionally, this research only considered a grid environment where all edges have the same cost. To create more realistic scenarios, it would be valuable to test the algorithms on grids with varied costs or on environments designed to resemble an even more realistic setting, perhaps even in 3D. Unfortunately, we did not have time to explore these aspects, but they would be interesting directions for future work.

Chapter 6

Conclusion

This study compared custom implementation of LPA* and D* Lite on a dynamic 2D grid with varying numbers of agents and dynamic obstacles. While LPA* performed well in less demanding scenarios, it struggled to handle frequent environmental changes, had slower replanning time, and was less effective at managing multiple agents compared to D* Lite. It is also important to note that the obtained mean path length and computation time are based only on successful trials. Since LPA* failed more often in complex scenarios, its averages are biased toward easier cases. D* Lite, despite having higher average values for both path length and computation time, is more reliable as it was able to produce results even in highly dynamic environments.

Additionally, although D* Lite requires initial knowledge of the map and cannot operate in completely unknown environments, it consistently computed paths and repaired broken routes efficiently. This enabled it to find valid paths more often than LPA*, even as the environment became increasingly complex and unrecognizable in comparison to the initial map.

In summary, this study revealed that D* Lite's main strength lies in its ability to quickly replan during environmental changes. An implementer should therefore consider D* Lite if there is likely to be significant congestion and frequent environmental changes. However, if the scenario is simpler with fewer environmental changes, then LPA* is a simpler algorithm with strong enough performance that should not be underestimated.

References

- [1] A. Moniz and B.-J. Krings, “Robots Working with Humans or Humans Working with Robots? Searching for Social Dimensions in New Human-Robot Interaction in Industry,” *Societies*, vol. 6, no. 3, p. 23, Aug. 2016, doi: <https://doi.org/10.3390/soc6030023>.
- [2] Y. Sazaki, H. Satria, and M. Syahroyni, “Comparison of A* and dynamic pathfinding algorithm with dynamic pathfinding algorithm for NPC on car racing game,” *IEEE Xplore*, Oct. 01, 2017. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8272918>
- [3] Z. Abd Algfoor, M. S. Sunar, and H. Kolivand, “A Comprehensive Study on Pathfinding Techniques for Robotics and Video Games,” *International Journal of Computer Games Technology*, vol. 2015, pp. 1–11, 2015, doi: <https://doi.org/10.1155/2015/736138>.
- [4] B. Jiang, “Traversing graphs in a paging environment, BFS or DFS?,” *Information Processing Letters*, vol. 37, no. 3, pp. 143–147, Mar. 2002, doi: [https://doi.org/10.1016/0020-0190\(91\)90034-F](https://doi.org/10.1016/0020-0190(91)90034-F).
- [5] L. Liu, X. Wang, X. Yang, H. Liu, J. Li, and P. Wang, “Path planning techniques for mobile robots: Review and prospect,” *Expert Systems with Applications*, vol. 227, pp. 120254–120254, Oct. 2023, doi: <https://doi.org/10.1016/j.eswa.2023.120254>.
- [6] M.-K. Ng, Y.-W. Chong, Kwang-Man Ko, Y.-H. Park, and Y.-B. Leau, “Adaptive path finding algorithm in dynamic environment for warehouse robot,” *Neural Computing and Applications*, vol. 32, no. 17, pp. 13155–13171, Feb. 2020, doi: <https://doi.org/10.1007/s00521-020-04764-3>.
- [7] Z. Ren, Sivakumar Rathinam, Maxim Likhachev, and Howie Choset, “Multi - Objective Path-Based D* Lite,” *IEEE robotics and automation letters*, vol. 7, no. 2, pp. 3318–3325, Apr. 2022, doi: <https://ieeexplore.ieee.org/document/9697421>.
- [8] S. Koenig and M. Likhachev, “D* Lite.” *AAAI*, 2002. <https://idm-lab.org/bib/abstracts/papers/aaai02b.pdf>.
- [9] M. Likhachev and S. Koenig, “A Generalized Framework for Lifelong Planning A* Search.” *AAAI*, 2005. https://www.cs.cmu.edu/~maxim/files/glpa_icaps05.pdf.

- [10] J. Jin, Y. Zhang, Z. Zhou, M. Jin, X. Yang, and F. Hu, "Conflict-based search with D* lite algorithm for robot path planning in unknown dynamic environments,". ScienceDirect. <https://doi.org/10.1016/j.compeleceng.2022.108473>.
- [11] C. Ju, Q. Luo, and X. Yan, "Path Planning Using an Improved A-star Algorithm IEEE Xplore," *ieeexplore.ieee.org*. <https://ieeexplore.ieee.org/stamp/ stamp.jsp?tp=&arnumber=9296641>.
- [12] F. Duchoň, D. Huňady, M. Dekan, and A. Babinec, "Optimal Navigation for Mobile Robot in Known Environment," *Applied Mechanics and Materials*, vol. 282, pp. 33–38. <https://doi.org/10.4028/www.scientific.net/amm.282.33>.
- [13] F. Duchoň *et al.*, "Path Planning with Modified a Star Algorithm for a Mobile Robot," *Procedia Engineering*. <https://doi.org/10.1016/j.proeng.2014.12.098>.
- [14] S. Koenig, M. Likhachev, and D. Furcy, "Lifelong Planning A*," *Artificial Intelligence*, vol. 155, no. 1–2, pp. 93–146, May 2004, doi: <https://doi.org/10.1016/j.artint.2003.12.001>.
- [15] A. Stentz, "The D* Algorithm for Real-Time Planning of Optimal Traverses Robotics " Institute Carnegie Mellon University," Robotics Institute Carnegie Mellon University, Sep. 13, 2017. <https://www.ri.cmu.edu/publications/the-d-algorithm-for-real-time-planning-of-optimal-traverses/>.
- [16] C. Hernández, J. Baier, R. Asin. "View of Making A* Run Faster than D*-Lite for Path-Planning in Partially Known Terrain," *Aaai.org*, 2025. <https://ojs.aaai.org/index.php/ICAPS/article/view/13675/13524>.
- [17] J. Alkazzi and K. Okumura, "A Comprehensive Review on Leveraging Machine Learning for Multi-Agent Path Finding, " *IEEE access* , pp. 1–1, Jan. 2024, doi: <https://doi.org/10.1109/access.2024.3392305>.

Appendix A

The source code used in the experiment is available on Github:

<https://github.com/Bik19/Da150x/tree/main>

